

Design Document: Production Machine Learning System

Project: House Prices Prediction API
Course: Machine Learning Model Engineering
Student: Somya Jaiswal
Student ID: 2025EM1100101
Repository: https://github.com/somj57/2025EM1100101_Somya_Jaiswal_Machine_Learning_Model_Engineering
Date: July 29, 2026

1. Problem Definition & Metrics

1.1 Use Case and Business Problem

The objective of this system is to predict continuous house sale prices based on various structural and locational attributes. The intended end-users are prospective buyers and sellers on a real estate web platform who require immediate, fair-market valuations when exploring properties or listing their homes.

1.2 Evaluation Metrics

The primary evaluation metric chosen for this regression task is **Root Mean Squared Error (RMSE)**.

- **Justification:** In real estate pricing, predicting a house's value incorrectly by \$100,000 is significantly more detrimental to user trust than being off by \$10,000 ten times. RMSE penalizes larger errors exponentially more than Mean Absolute Error (MAE), aligning perfectly with the business goal of avoiding massive overvaluations or undervaluations.

2. Data & Feature Design

2.1 Data Description and Ingestion

The dataset utilizes the Ames Housing dataset, containing detailed property attributes. The target label is `SalePrice`. The data pipeline features a micro-batch ingestion script (`ingest.py`) designed to simulate a production environment where new daily CSV drops are appended to a master raw training table.

Ingestion Evidence:

```
[2026-07-29 15:31:53.434199] Ingesting data from https://raw.githubusercontent.com/.../train.csv...  
[2026-07-29 15:31:56.727686] Ingested 1460 rows. Saved to data/raw/training_table.csv
```

2.2 Feature Engineering

To improve predictive power, sparse or closely related variables were synthesized into five non-trivial features:

1. **House_Age:** $\text{YrSold} - \text{YearBuilt}$
2. **Remodel_Age:** $\text{YrSold} - \text{YearRemodAdd}$
3. **Total_Square_Footage:** Aggregation of `TotalBsmtSF`, `1stFlrSF`, and `2ndFlrSF`.
4. **Total_Bathrooms:** Weighted sum of full and half baths ($\text{FullBath} + 0.5 * \text{HalfBath} + \text{BsmtFullBath} + 0.5 * \text{BsmtHalfBath}$).
5. **Has_Pool:** A binary flag mapping $\text{PoolArea} > 0$ to 1, and 0 otherwise.

During the feature engineering phase, it was determined that retaining the original component columns (e.g., `1stFlrSF`, `2ndFlrSF`) introduced unnecessary dimensionality. Therefore, the pipeline explicitly removes these original component columns before training, passing forward only the derived, high-utility features to the model.

2.3 Handling Training-Serving Skew

To completely eliminate training-serving skew, all feature engineering logic is encapsulated inside a custom Scikit-Learn `TransformerMixin` class (`HouseFeatureEngineer`). This class is bundled directly into the `sklearn.pipeline.Pipeline` object. Consequently, the exact same Python code that transforms the batch CSV data during offline training automatically executes on the real-time JSON payload during online serving.

3. Model Choice & Evaluation

3.1 Model Selection

- **Baseline Model:** A Dummy Regressor predicting the mean sale price of the training set.
- **Candidate Model:** A Random Forest Regressor (`n_estimators=100`).
- **Justification:** Random Forests natively handle non-linear relationships and are robust to outliers, making them highly effective for tabular real estate data without requiring extensive one-hot encoding or scaling of continuous variables.

3.2 Offline Evaluation Harness

The training script splits the ingested data (80/20 train/test split). A strict deployment guardrail is implemented: the candidate model is only serialized and saved to the model registry (`models/v1/`) if its RMSE is strictly lower than the baseline.

Evaluation Evidence:

```
Baseline RMSE: $87,619.03
Candidate RMSE: $32,600.14
Candidate model is better. Promoting to production!
```

4. Serving & Inference Pattern

4.1 Inference Architecture

The model is served using an **Online API (Request-Response)** pattern built with FastAPI.

- **Justification:** The primary consumers of this system are human users interacting with a web interface (e.g., adjusting a slider for "Bathrooms" to see how it affects the house price). A human is waiting for the result; therefore, a batch processing pattern is entirely unsuitable. The acceptable latency threshold for a seamless UI experience is $< 200\text{ms}$.

4.2 API Design and Performance

The `/predict` endpoint accepts a JSON payload validated via Pydantic and returns the predicted price alongside the `model_version` to ensure traceability.

Latency Testing Evidence (100 requests):

```
Total Requests: 100
Average Latency: 5.94 ms
P95 Latency: 14.11 ms
```

The P95 latency of 14.11 ms proves the system is highly performant and easily meets the sub-200ms requirement for interactive web applications.

5. Monitoring, Data Quality & Retraining

5.1 Monitoring Plan

A comprehensive production system requires monitoring across three pillars:

1. **Infrastructure Metrics:** Track API endpoint average latency, P95 latency, and HTTP 5xx error rates to ensure server health.
2. **Data/Feature Metrics:** Monitor the percentage of missing values (null counts) in incoming JSON requests and track statistical drift on continuous features like `Total_Square_Footage`.
3. **Business/Model Metrics:** Once true sale prices are realized and fed back into the system, track the moving average of the RMSE to detect model decay.

5.2 Drift Check Implementation

A lightweight drift-detection script was implemented to compare the mean of `TotalBsmtSF` between the original training data and any newly ingested batch.

Drift Check Evidence:

```
Training Mean BsmtSF: 1057.43 | New Batch Mean: 1057.43
```

5.3 Retraining Trigger Logic

Retraining is governed by the following pseudo-code logic, evaluated by an orchestration tool (e.g., Airflow):

1. **Time-based:** If `days_since_last_train` ≥ 30 , trigger scheduled retraining.
2. **Data-driven:** If `drift_ratio` > 0.15 (15% deviation in key features), trigger immediate retraining.
3. **Performance-driven:** If labeled feedback shows `recent_RMSE` $>$ `baseline_threshold`, trigger retraining.

5.4 Incident Scenario: Upstream Schema Change

Scenario: A data provider unexpectedly changes the reporting unit of `TotalBsmfSF` from square feet to square meters, causing a sudden drop in the numerical values.

Detection: The automated daily drift check (Section 5.2) will immediately flag a $>15\%$ deviation in the feature mean and fire a critical alert to the MLOps team.

Response: The automated retraining pipeline is paused to prevent the model from learning corrupted data. The engineering team rolls back to the previous stable model version (via the tracked `model_version` API tag), writes a conversion mapping function in the ingestion script, and deploys a hotfix.

6. System Architecture

SYSTEM ARCHITECTURE FLOW

1. Data Pipeline (Batch)

Daily CSV Drops



ingest.py

Data Ingestion Script



Appends

Raw Data Storage (data/raw/)

2. Model Training Pipeline

Scikit-Learn Pipeline
(Feature Eng + Random Forest)



train.py

Offline Evaluation
(Baseline vs Candidate RMSE)



If Candidate is Better

Model Registry (models/v1/)

3. Online Serving

FastAPI Endpoint (/predict)



serve.py (Loads .joblib)

End User Web App (JSON UI)

4. Monitoring & MLOps

Monitoring Logs & Drift Check
(monitor.py)



Alert on > 15% Drift

Trigger Retraining Pipeline